



Jira Sprinkles



Multi-Agent Code Generation & Self-Repair System



Agenda

1. Introduction and Problem Context
2. System Architecture and Workflow
3. AI Integration, Scalability, and Value



Introduction and Problem Context



Problem And Impact

Where teams lose time today

- Build and test breaks force repeated investigation and local reproduction.
- Context is scattered across logs, code changes, and ticket comments.

Why it is costly

- Ticket updates and evidence gathering create avoidable waiting and rework.
- Hand-offs slow delivery and reduce confidence in release readiness.

Impact of autonomous repair

- Automated diagnosis and fixes shorten cycles from failure to validation.
- Consistent results improve delivery quality and support continuous delivery.



Why It Matters

Release speed and effort

- Manual triage and rework slow delivery and consume engineering time.
- Failures often require repeated investigation, retesting, and coordination.

Autonomous recovery loop

- The pipeline generates Java service code and JUnit tests automatically.
- When builds fail, it diagnoses causes, applies fixes, and re-runs tests.

Operationally connected

- Integrates with JIRA, Maven, Git, Jenkins, and health checks for validation.
- Creates clearer ownership through ticket evidence and consistent status updates.

Outcome for stakeholders

- Faster recovery with fewer handoffs and lower operational friction overall.





System Architecture and Workflow

System Architecture



Orchestration layer

- A Flask app runs the workflow engine, configuration, and live run tracking.
- Progress events are emitted per step for dashboard visibility and auditability.



Workflow execution model

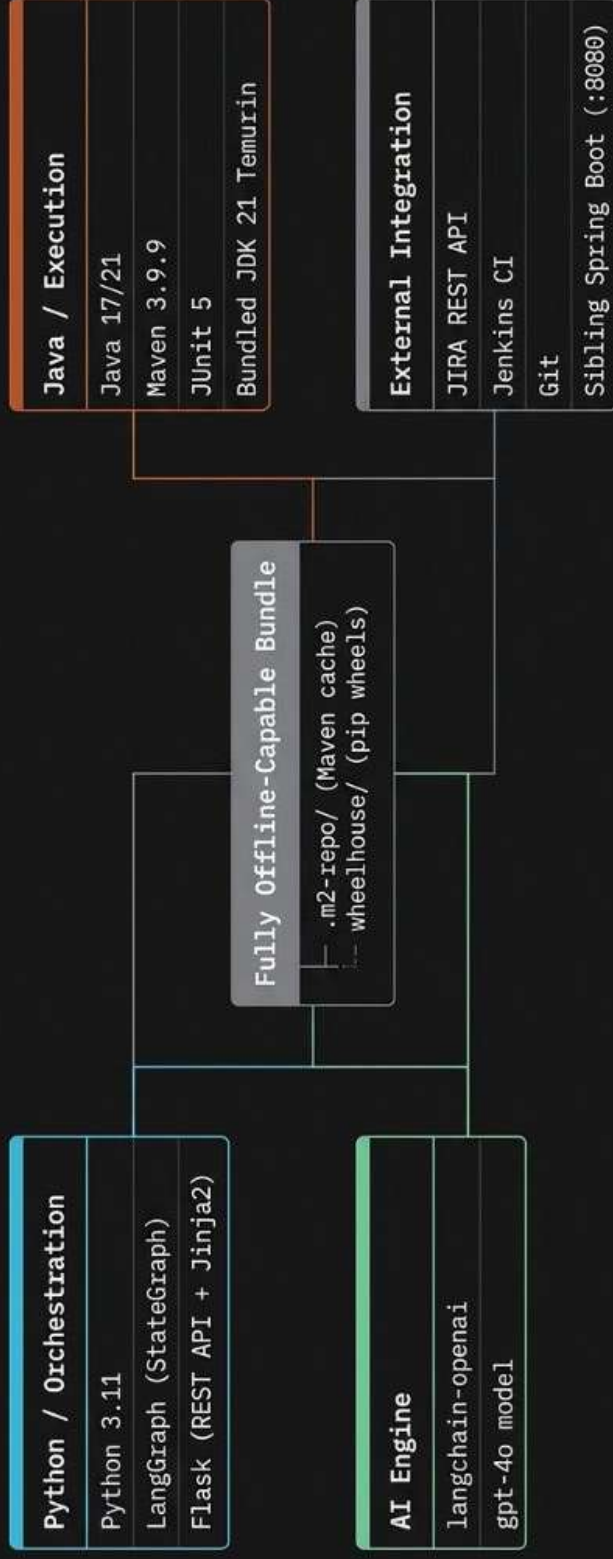
- LangGraph coordinates 14 steps using a single shared **AgentState** contract.
- Router decisions read state fields to choose success paths or repair loops.



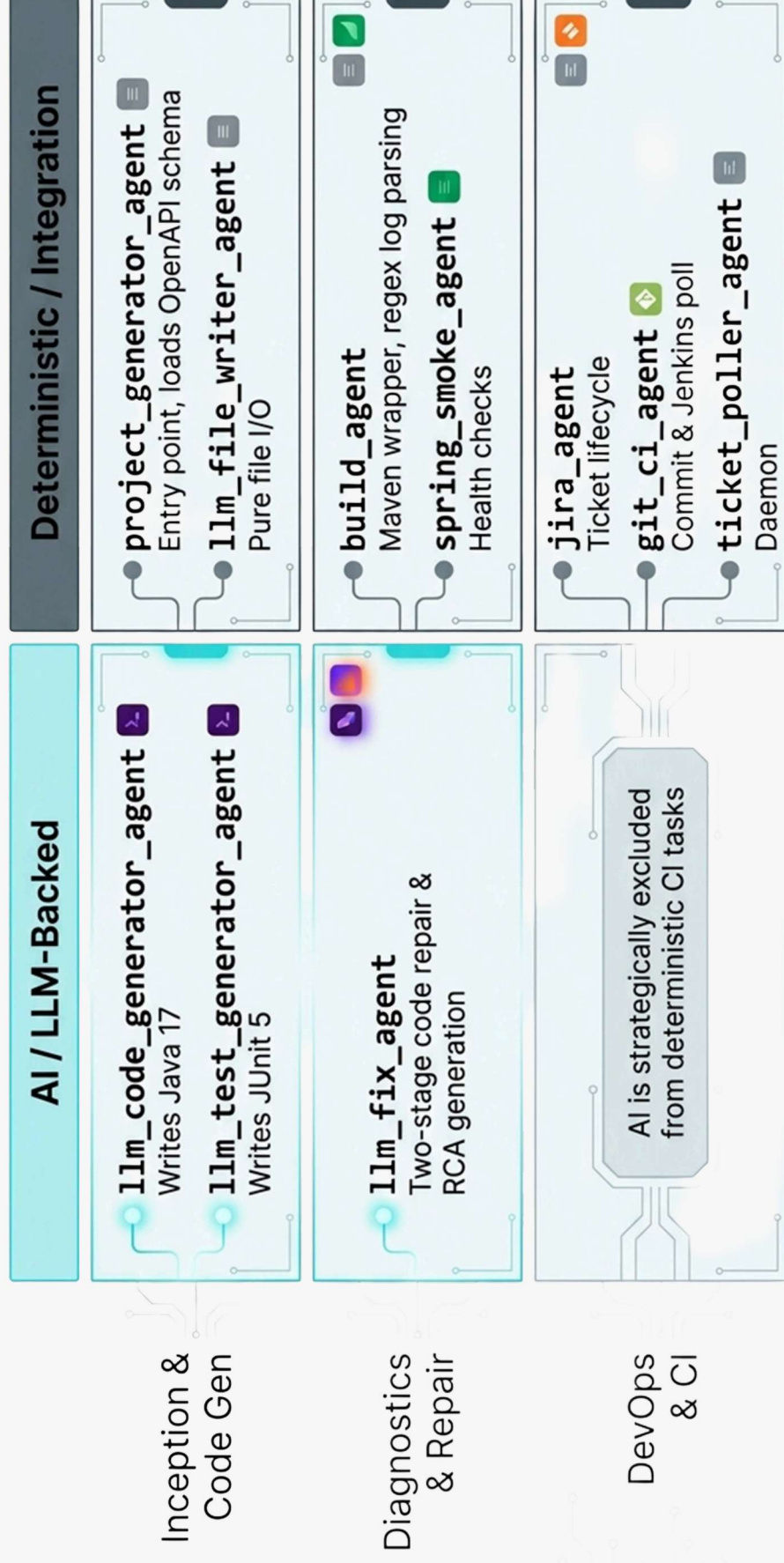
Enterprise integrations

- OpenAI gpt-4o model generates code, tests, and repair suggestions when failures occur.
- Maven validates builds; JIRA, Git, Jenkins, and Spring checks close the loop.

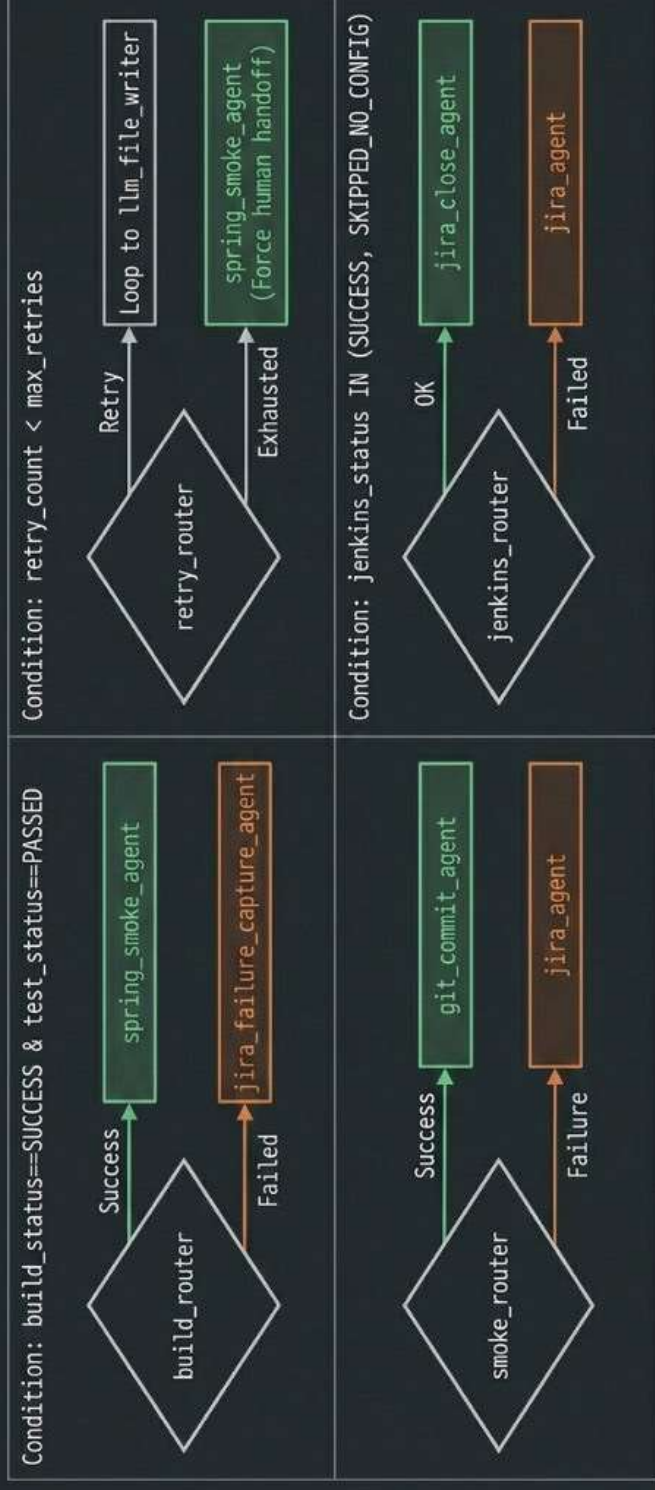
The Engineering Stack and Execution Environment



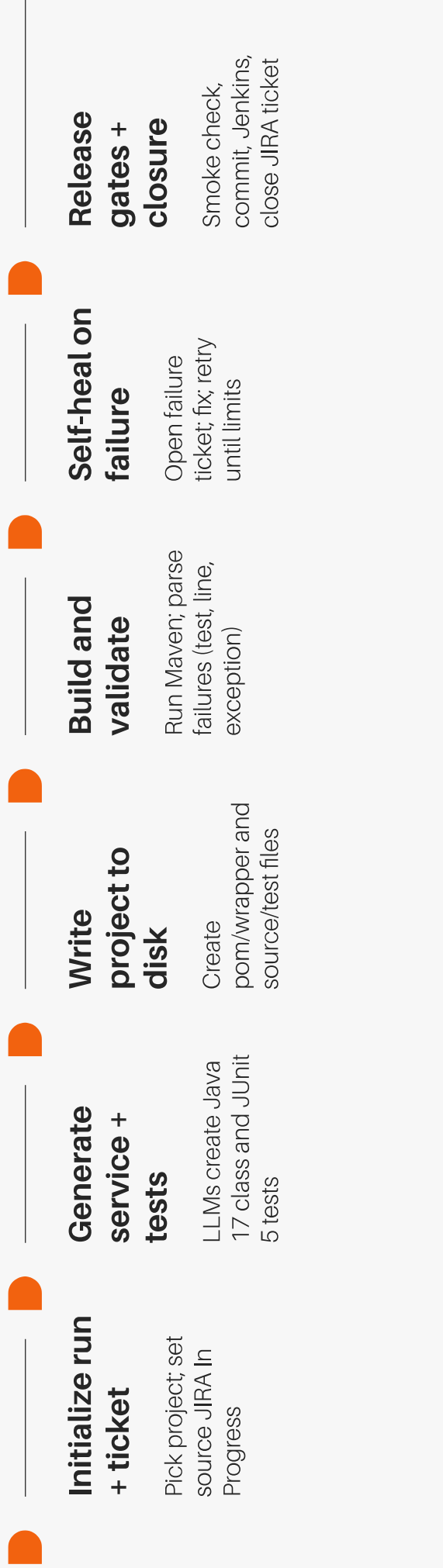
> Execution: ./start.ps1 natively handles runtime caching without external internet dependencies (beyond OpenAI API).



Deterministic Routing Logic



End-to-End Flow





LLM Inputs & Outputs

Code generator: inputs

- Uses project name, domain, and API schema summary as context.
- Optionally reads a bug trigger to inject a known failure type.

Code generator: outputs

- Produces a Java 17 service class with one public method.
- Returns source text plus metadata for later build diagnostics.

Test generator: inputs and outputs

- Takes generated service code and project details to design tests.
- Outputs a JUnit 5 test class aligned to the public method.

Fix agent: expected outputs

- Delivers corrected source and, if needed, updated failing tests.
- Adds an RCA comment and a short summary of the applied fix.





Self-Healing Control Loop

Evidence-first ticketing

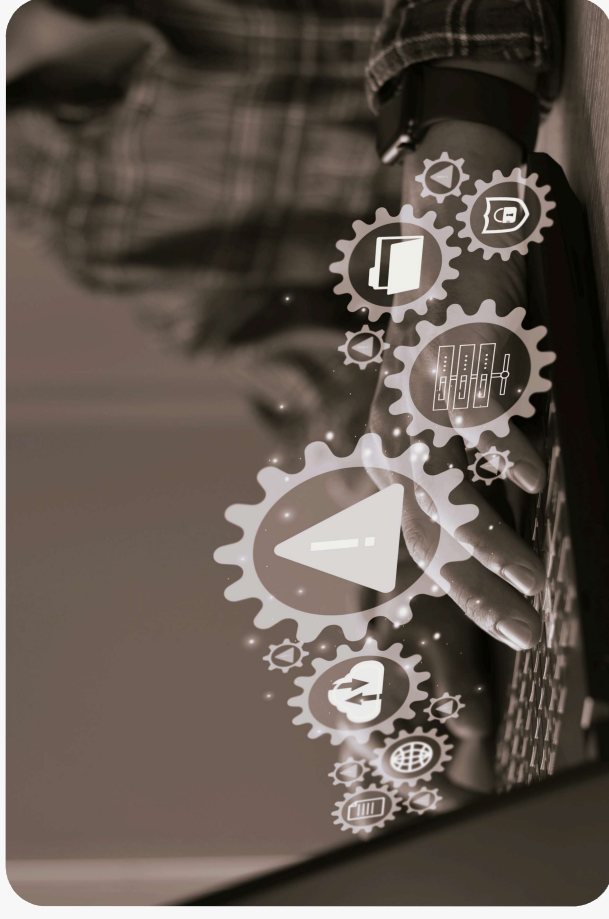
- On failure, a JIRA ticket is created with logs and code context.
- This preserves repro details before any automated repair changes files.

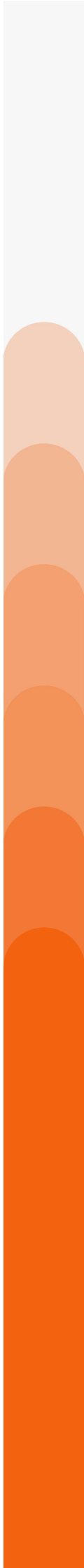
Two-stage repair loop

- Retry 0 applies a minimal **source-only fix** to reduce churn.
- Later retries may adjust tests and include an RCA comment.

Predictable routing and governance

- Routers use state fields to choose retry, proceed, or handoff.
- Deterministic branching makes actions auditable and easier to govern.





AI Integration, Scalability, and Value



AI Depth And Safety



AI where it adds judgment

- LLMs generate the Java service, JUnit tests, and repairs.
- Repairs include a clear RCA comment for stakeholders.



Deterministic control and evidence

- Non-AI nodes run builds, write files, and manage ticket states.
- Build parsing captures exception type and failing test details.



Risk controls and fallbacks

- Two-stage fix: source-only first, then source plus tests.
- Deterministic fallbacks handle LLM failures or incomplete outputs.



Scalability & Reuse



Modular building blocks

- Specialized agents can be reused for generation, repair, and CI steps.
- Clear AI vs deterministic roles make components easier to swap.



Extensible state contract

- A shared AgentState carries context without redesigning integrations.
- Router decisions stay predictable because they read the same fields.



Enterprise-ready operations

- Ticket polling enables recurring, cross-application failure handling.
- Tracked node events improve traceability, audits, and maintainability.



Why It Stands Out

Closed-loop delivery, not just generation

- It automatically rebuilds, detects failures, repairs, and re-validates outcomes.
- Typed workflow state keeps decisions consistent and auditable end-to-end.

Works with existing engineering systems

- Integrates with JIRA, Git, Jenkins, Maven, and Spring health checks.
- Tickets capture evidence and progress without added manual coordination.

Clear stakeholder value and alignment

- Faster visibility into failures plus cleaner RCA for accountability.
- Supports automation and engineering productivity goals at scale.



Operational AI, at scale

Autonomous, end-to-end recovery

- One workflow generates code, validates it, and auto-repairs failures.
- JIRA, Git, Jenkins, Maven, and health checks stay in sync.

Measurable efficiency gains

- Cuts manual debugging loops by capturing evidence and RCA automatically.
- Deterministic routing makes outcomes repeatable and auditable.

Deep AI, controlled execution

- AI focuses on generation and repair; deterministic agents run builds and CI.
- Fallback fixes reduce risk when LLM outputs are incomplete.

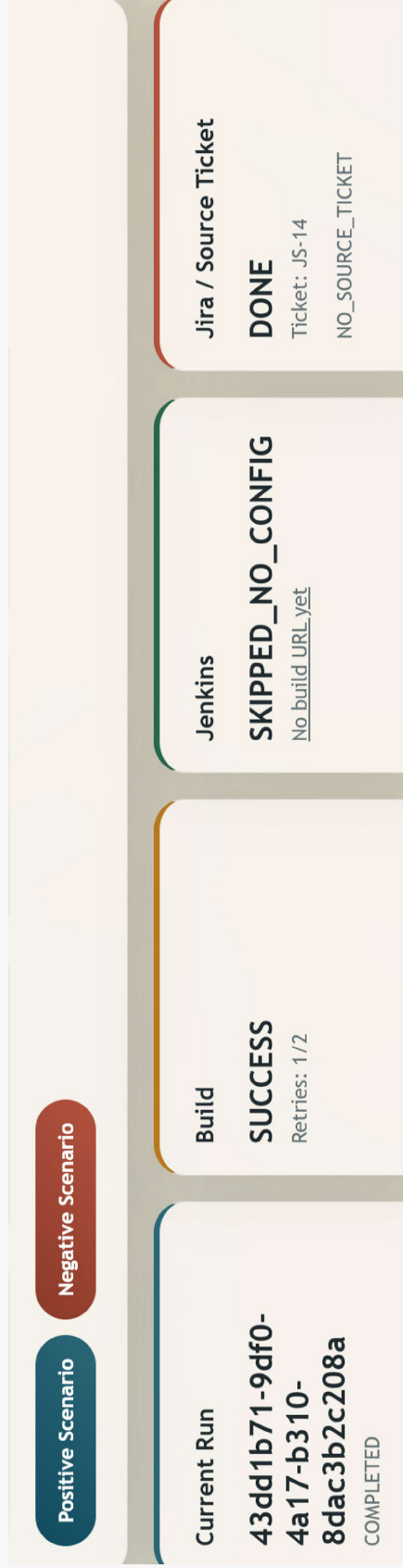
Ready to adopt and scale

- Modular agents and a shared state contract simplify extension to new uses.

Positive Scenario (No issues in the code generated)



Negative Scenario (Runtime issues in the code generated)



Jira Info

Jira Ticket

JS-14

[JS] ArrayIndexOutOfBoundsException in InventoryService.evaluateRiskyPath() (inventory-management-system)

WHAT FAILED

A newly added corner-case test detected a runtime defect in the generated Exception type : ArrayIndexOutOfBoundsException
Exception detail: org.opentest4j.AssertionFailedError: Unexpected exception

WHERE (LOCATION)

Service class : InventoryService

Service method : evaluateRiskyPath

Source file : src/main/java/com/demo/InventoryService.java

Failing test : InventoryServiceEdgeCaseGeneratedTest.shouldHandleInjected

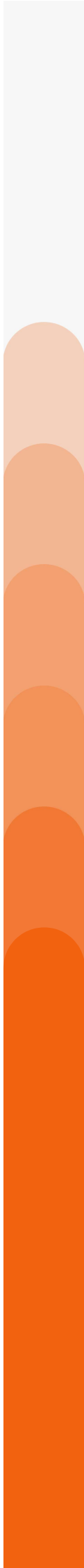
Test file : src/test/java/com/demo/InventoryServiceEdgeCaseGeneratedTest

☐	Work	Status	Resolution	Created	
☐	JS-14 [JS] ArrayIndexOutOfBoundsException in Inventor...	DONE ▾	Done	Jun 12, 2026, 1:14 PM	

21.



Questions & Answers



 Thank you
